	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 9

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 3 de Noviembre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**
1.1. Planteamiento del problema 2
1.2. Motivación 2
1.3. Objetivos 2

2. Marco Teórico **2**
2.1. Lenguaje de Modelado Unificado (UML) 2
2.2. Diseño estático o de estructura 3
2.3. Diagrama de clases 3
2.4. Diseño dinámico o de comportamiento 3
2.5. Diagrama de secuencia 3

3. Desarrollo **3**

4. Resultados **5**
4.1. Diagramas prática 5-8 5
4.2. Diagramas Hospital 13

5. Conclusiones **14**

1. Introducción

1.1. Planteamiento del problema

La práctica número 9 consiste en representar, mediante diagramas de Lenguaje de Modelado Unificado (UML), los sistemas desarrollados en sesiones anteriores. Específicamente, el problema a resolver es elaborar el diagrama de clases (incluyendo su cardinalidad) y el diagrama de secuencias para:

- **Prácticas de la 5 a la 8.** Modelar la estructura estática y la interacción dinámica de los ejercicios de Programación Orientada a Objetos resueltos en dichas prácticas (tales como el sistema de empleados, manejo de fechas, jerarquía de figuras geométricas y cálculo de la huella de carbono).
- **Proyecto de POO (Etapa 2).** Documentar formalmente la estructura de clases y el flujo de mensajes del sistema hospitalario construido para la segunda etapa del proyecto semestral.

1.2. Motivación

Es necesario dar solución a este problema porque comprender y aplicar el UML es fundamental en la ingeniería de software para visualizar, especificar y documentar el diseño de un sistema. Al pasar del código a los diagramas, se verifica que la jerarquía de herencia, las relaciones de composición y la interacción entre objetos se hayan implementado con una correcta abstracción y encapsulamiento.

1.3. Objetivos

Lo que se espera obtener al darle solución a esta práctica es:

- Utilizar UML como herramienta para documentar y diseñar soluciones de software orientadas a objetos.
- Construir diagramas de clases detallados con cardinalidad para definir la estructura estática de cada sistema.
- Elaborar diagramas de secuencias para trazar el flujo de ejecución, la línea de vida y los mensajes intercambiados entre los objetos en un instante de tiempo.

2. Marco Teórico

2.1. Lenguaje de Modelado Unificado (UML)

El Lenguaje de Modelado Unificado (UML) es un lenguaje gráfico que permite visualizar, especificar y documentar cada una de las partes que comprende el desarrollo de software. [1]

Los diagramas UML permiten modelar aspectos conceptuales como procesos de negocios o funcionalidades de un sistema, cumpliendo con los siguientes objetivos: visualizar, especificar, construir y documentar. [2]

UML está compuesto por 3 bloques generales de formas: elementos, relaciones y diagramas. [2]

2.2. Diseño estático o de estructura

Los diagramas estáticos o estructurales aportan una visión fija del sistema. [1]

2.3. Diagrama de clases

Un diagrama de clases permite modelar las características de las clases que componen al sistema. [1]

Dentro de cada clase se pueden visualizar los atributos y métodos. [2]

El conjunto de clases permite observar las relaciones que existen entre ellas, indicando explícitamente su cardinalidad (por ejemplo, relaciones de 1 a muchos). [1]

2.4. Diseño dinámico o de comportamiento

Los diagramas dinámicos o de comportamiento permiten visualizar la comunicación entre elementos del sistema para un proceso específico. [2]

2.5. Diagrama de secuencia

Los diagramas de secuencia muestran una interacción ordenada de eventos, visualizando los objetos (instancias de clases) participantes en cada interacción. [1]

Permiten ver los mensajes que intercambian entre ellos, trazando claramente el flujo de la llamada y la respuesta a lo largo de una línea de tiempo para llevar a cabo una acción en el sistema. [2]

3. Desarrollo

Prácticas 5 a 8

1. **Análisis del código fuente.** Se revisan los archivos Java desarrollados en las prácticas 5, 7 y 8 para identificar las clases, atributos, métodos y constructores implementados.

2. **Identificación de relaciones.** Se analiza cómo interactúan las clases entre sí, identificando la herencia en las figuras geométricas y la implementación de interfaces en el cálculo de la huella de carbono.

3. **Diseño de los diagramas de clases.** Se elaboran los diagramas estáticos representando cada clase con sus respectivos modificadores de acceso, tipos de datos y firmas de métodos.

4. **Asignación de cardinalidades.** Se añaden las multiplicidades numéricas en las asociaciones, estableciendo explícitamente cuántos objetos se relacionan entre sí (por ejemplo, 4 puntos para un cuadrilátero).

5. **Trazado de líneas de vida.** Para el comportamiento dinámico, se identifican los objetos instanciados (como `emp1` o `fecha`) y se dibujan sus líneas de vida en los diagramas de secuencia.

6. **Modelado de mensajes.** Se representan las llamadas a los métodos (como el cálculo de áreas o modificación de salarios) indicando el flujo de ejecución mediante flechas de llamada y de retorno.

7. **Verificación visual.** Se contrastan los diagramas generados contra el código original para confirmar que modelen exactamente la lógica de los programas.

Proyecto Etapa 2: Sistema Hospitalario

1. **Revisión de la arquitectura.** Se examina la estructura del sistema hospitalario, identificando superclases como `PersonalMedico` y `Paciente`, así como sus clases derivadas.

2. **Construcción del diagrama de clases.** Se esquematiza la jerarquía completa, integrando la interfaz `AtencionMedica` y la clase `Sistema` que administra las colecciones de objetos.

3. **Definición de restricciones.** Se plasma la cardinalidad en las dependencias, especificando restricciones de negocio como el límite de pacientes que puede atender un médico.

4. **Identificación del flujo principal.** Se determina el orden cronológico de los procesos de la etapa 2: registro de usuarios, asignación de consultas y aplicación de tratamientos.

5. **Elaboración del diagrama de secuencia.** Se documenta la interacción temporal paso a paso desde el actor (Usuario) hasta los mensajes síncronos intercambiados entre las instancias del sistema.

6. **Comprobación de polimorfismo.** Se valida gráficamente que los mensajes deleguen correctamente la responsabilidad a los objetos instanciados en tiempo de ejecución.

7. **Cierre del trabajo.** Se exportan los diagramas completos desde la herramienta de modelado para integrarlos al reporte como evidencia del diseño.

4. Resultados

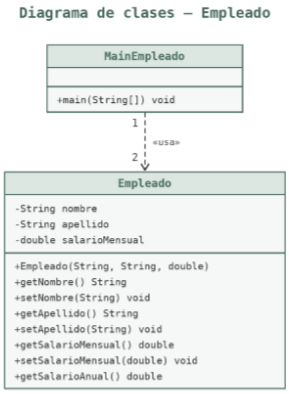
4.1. Diagramas prática 5-8

Practica 9_UML

Practica 5

Diagrama de clases con cardinalidad

Ejercicio 2



Ejercicio 3

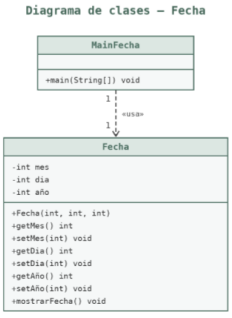


Figura 1: Diagramas

Diagrama de secuencias

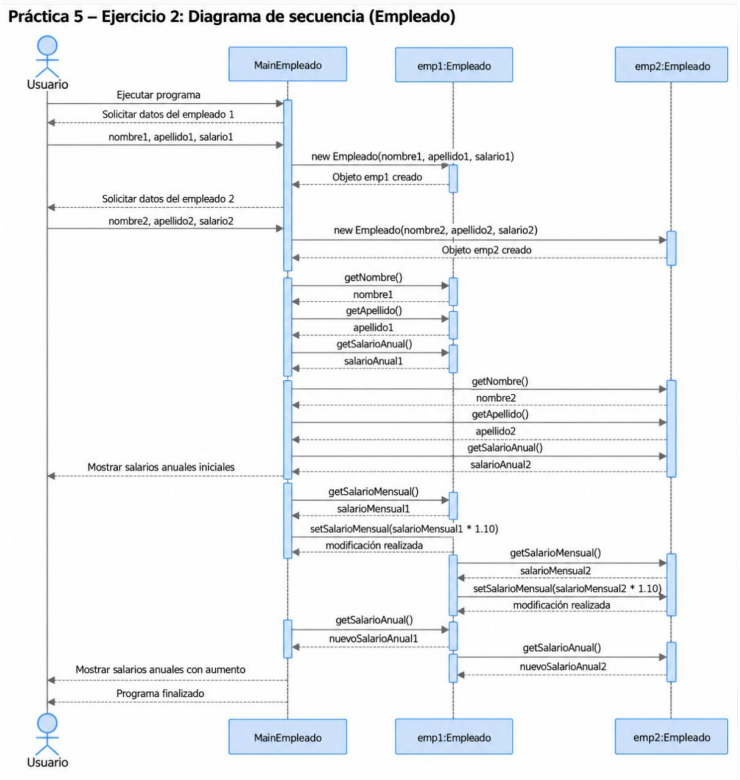
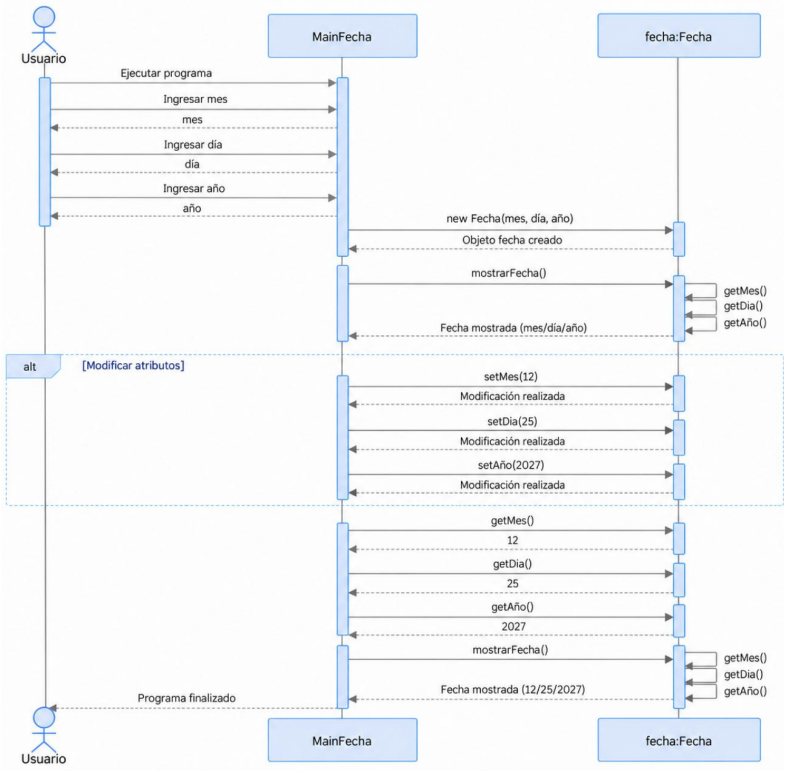


Figura 2: Diagramas.

Ejercicio 3_Fecha



Practica 7

Figura 3: Diagramas.

Diagrama de clases con cardinalidad

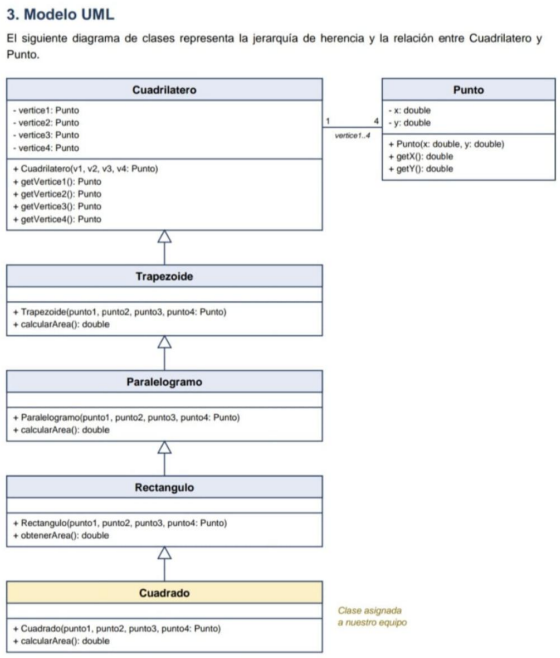
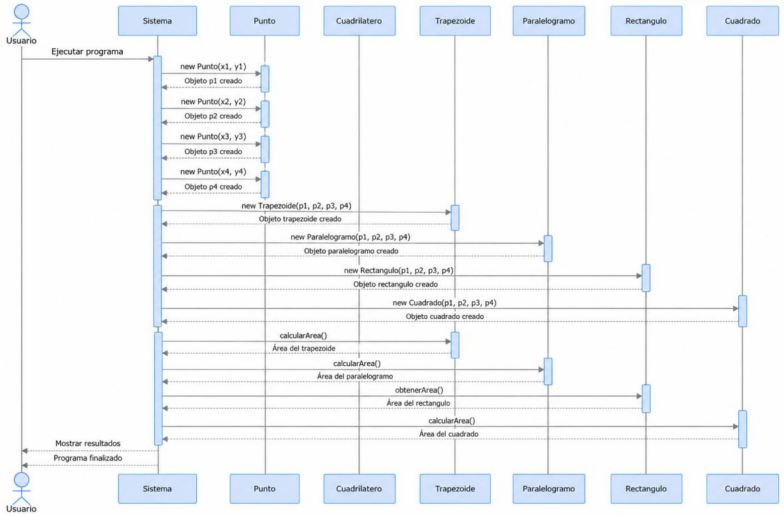


Diagrama de secuencias

Figura 4: Diagramas.



Practica 8

Figura 5: Diagramas.

Diagrama de clases con cardinalidad

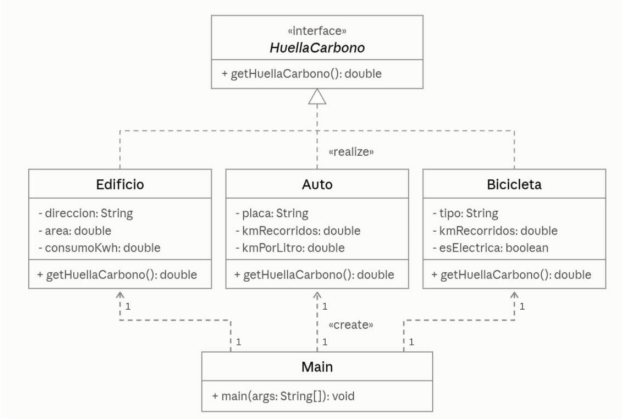
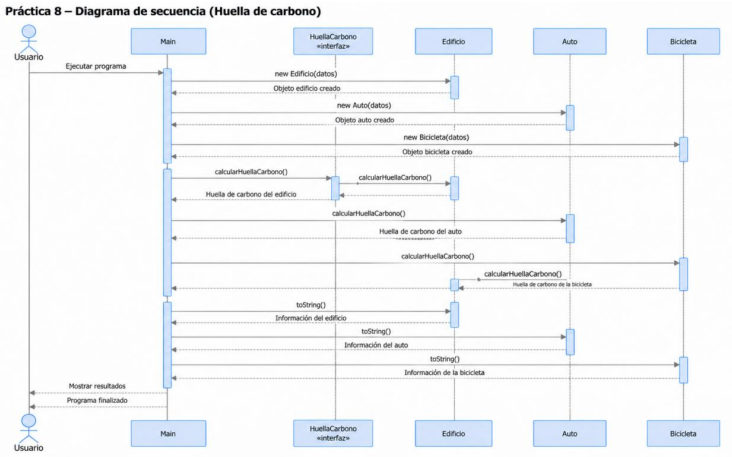


Diagrama de secuencias



Proyecto etapa 2

Diagrama de clases con cardinalidad

Figura 6: Diagramas.

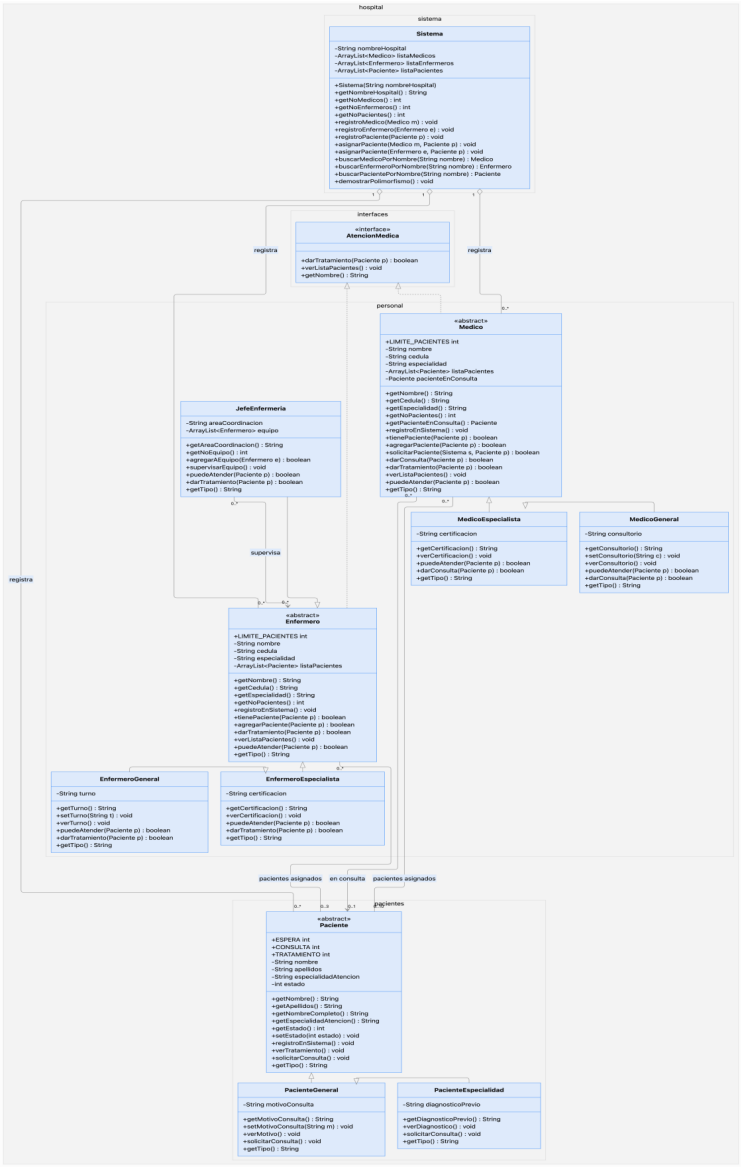


Diagrama de secuencias

Figura 7: Diagramas.

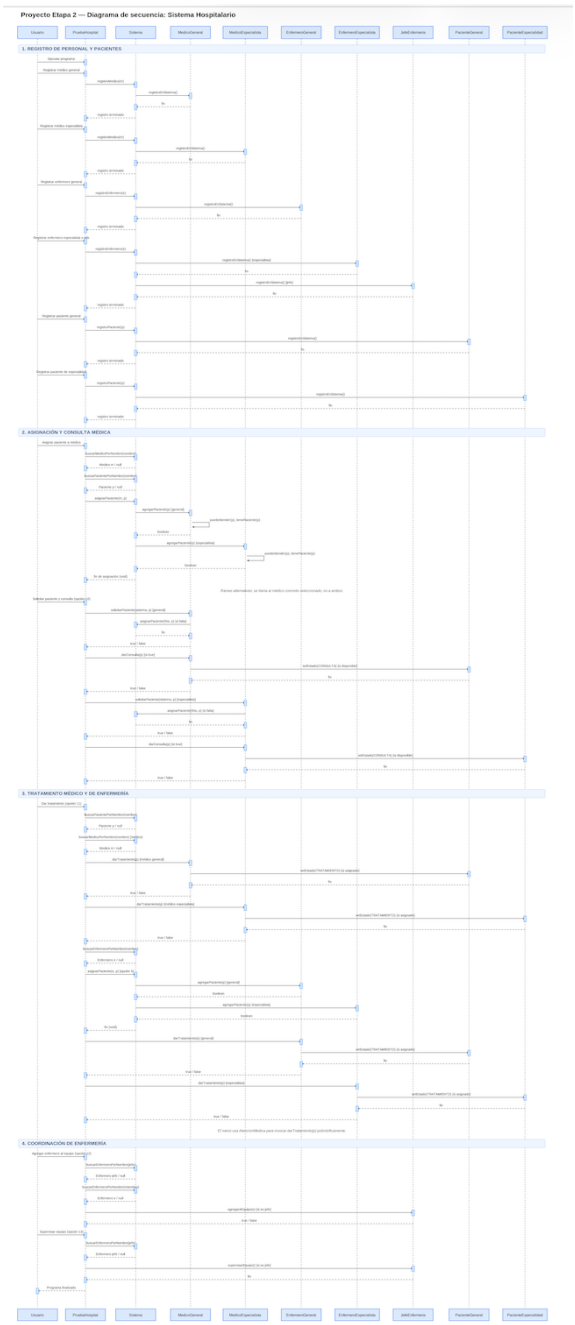


Figura 8: Diagramas.

4.2. Diagramas Hospital

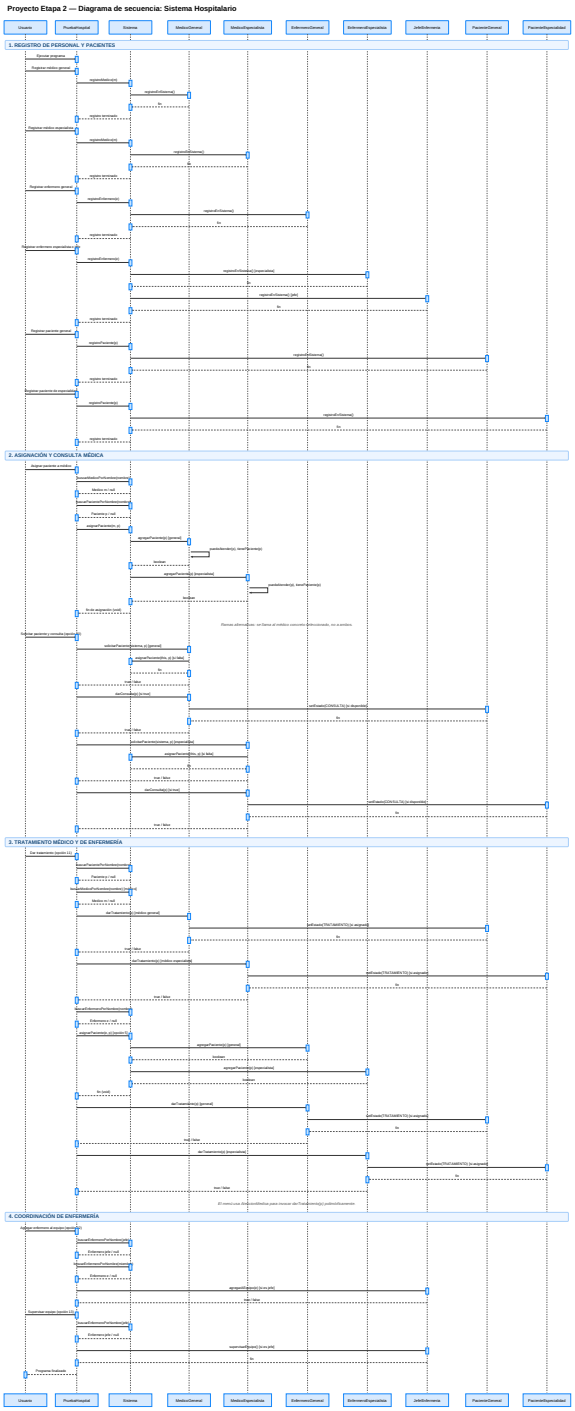


Figura 9: Diagrama de secuencia completo del Sistema Hospitalario.

5. Conclusiones

El análisis de los diagramas resultantes confirma la importancia de aplicar los conceptos teóricos de modelado UML para diseñar soluciones de software

- **Diseño estático:** Los diagramas de clases permitieron estructurar visualmente el sistema, garantizando la correcta aplicación del encapsulamiento, las jerarquías de herencia y la cardinalidad exacta en las relaciones entre entidades
- **Diseño dinámico:** Los diagramas de secuencia trazaron el flujo cronológico de los mensajes, lo que validó la interacción entre instancias en tiempo de ejecución y comprobó el funcionamiento del polimorfismo
- **Validación arquitectónica:** UML tradujo la abstracción teórica en planos concretos, paso indispensable para estructurar lógicamente sistemas de alta complejidad, como el proyecto hospitalario, antes de su codificación

Referencias

[1] M. A. Nakayama C. y J. A. Solano G., *Manual de prácticas del Laboratorio de Programación orientada a objetos*, 1.a ed. Ciudad de México: Facultad de Ingeniería, UNAM, 2017, pp. 142-150.

[2] Apuntes de clase, *Semana 9. Lenguaje de Modelado Unificado (UML)*, Pizarrón del Laboratorio de Programación Orientada a Objetos, Facultad de Ingeniería, UNAM.